



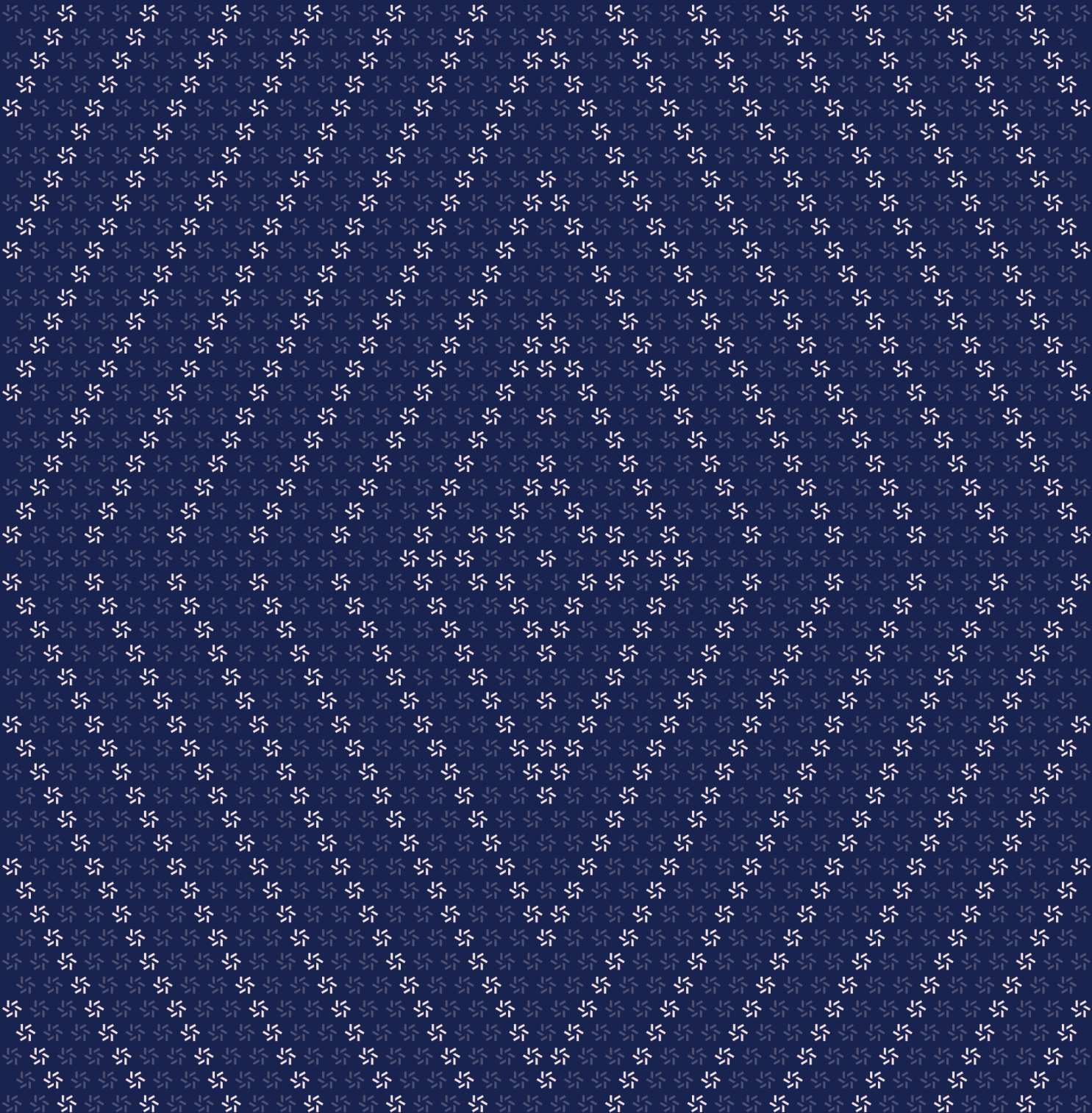
Prepared for
Evan Wang
Glen Rose
Tydro

Prepared by
Qingying Jie
Kuilin Li
Chongyu Lv
Zellic

August 4, 2026

Multiply

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	6
1.4. Results	6
2. Introduction	8
2.1. About Multiply	8
2.2. Methodology	8
2.3. Scope	10
2.4. Project Overview	10
2.5. Project Timeline	11
3. Detailed Findings	12
3.1. Counterfactual aToken poisoning blocks opening	12
3.2. Failed open () calls can strand prefunded raw collateral	16
3.3. SafeTransferLib treats calls to no-code addresses as successful ERC-20 calls	19
3.4. EMode transition checks are stricter than the underlying Aave rules	21
3.5. Active flash-loan callback commitments remain live during callback execution	23

4.	Discussion	27
4.1.	Discussion of registry pool configurability	27
4.2.	The exact-input life cycle leaves <code>ConversionLeg.maxAmountIn</code> redundant	29
5.	Threat Model	30
5.1.	Module: TydroGuardian	30
5.2.	Module: TydroMarketRegistry	33
5.3.	Module: TydroPositionAccount	44
5.4.	Module: TydroPositionAccountFactory	50
5.5.	Module: TydroVelodromeSlipstreamSwapRoute	53
6.	Assessment Results	59
6.1.	Disclaimer	59

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Tydro from July 17th to July 28th, 2026. During this engagement, Zellic reviewed Multiply's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is `TydroPositionAccount` decomposed at the right security boundaries? Would extracting authorization, transient callback state, Aave settlement, bounded route execution, or postcondition accounting materially improve architecture?
- Do ERC-1167 immutable-argument clones and EIP-1153 transient callback state provide sufficient isolation, deployment, and safety benefits to justify their complexity? Is there a materially simpler design that preserves deterministic identity, replay isolation, authenticated flashloan callbacks, and nonupgradability?
- Can `open` or `close` execute without a valid EIP-712 signature from the account's exact immutable owner?
- Can `setEModeCategory` or `claimATokens` be invoked by anyone other than the account's exact immutable owner?
- Can a signed intent be replayed across accounts, chains, or between the `open` and `close` type hashes? Is the EIP-712 domain computed against the ERC-1167 clone address rather than the implementation?
- Is there any way to skip, reuse, or reorder the nonce? Does the absence of an explicit intent-cancellation function create unacceptable bearer-intent or relayer griefing risk?
- Can `executeOperation` be invoked, spoofed, or reentered outside a live owner-authorized life cycle? Can an open callback context be used for `close`, or vice versa?
- Does the transient active/pending callback state prove that the genuine Aave callback ran to completion? Can callback state be forged, pre-seeded, reused, or left stale across transactions or accounts?
- Are `open` and `close` postconditions evaluated at the correct points relative to Aave's flash-loan life cycle, particularly mode-2 debt creation after the open callback and mode-0 principal/premium collection after the close callback? Can interest accrual, debt dust, premium calculation, or transaction ordering produce a successful life cycle outside the signed bounds?
- Are the exact-allowance and balance-delta checks sound for the supported `weETH` and `WETH` implementations?

- Can the account or route consume more than the exact signed input, retain an allowance, retain output, misreport output, or treat route return telemetry as proof of successful settlement?
 - Can registry policy epochs, route-local config epochs, config hashes, or routeStateHash become incorrectly synchronized or be bypassed? Can queued activation survive an intervening route, registry, risk-cap, ownership, or Guardian action when it should become stale?
 - Can route disablement, registry/config drift, oracle/venue divergence, or insufficient liquidity prevent an otherwise solvent owner from reducing risk or closing? Is a close-only fault-protection path warranted, and if so, what is the narrowest design that preserves owner authorization, bounded token movement, measurable settlement, and protection against oracle or route abuse?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

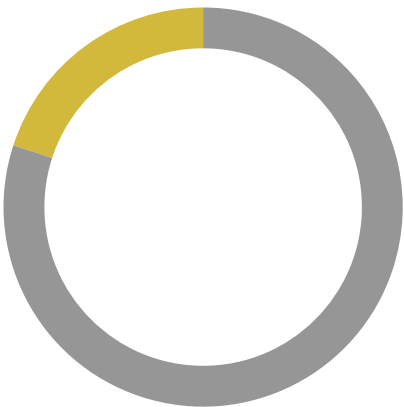
1.4. Results

During our assessment on the scoped Multiply contracts, we discovered five findings. No critical issues were found. One finding was of medium impact and the other findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Tydro in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	0
Informational	4



2. Introduction

2.1. About Multiply

Tydra contributed the following description of Multiply:

Multiply is a DeFi protocol for opening and closing leveraged/looped positions on Aave v3. The abstraction between position account and the routes represent the the future extendability of new routes and venues.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Multiply Contracts

Type	Solidity
Platform	EVM-compatible
Target	multiply-contracts-test
Repository	https://github.com/tydrohq/multiply-contracts-test ↗
Version	c2adc094a686d5ba1b4a191a8b703feba9adcaa9
Programs	src/**/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.3 person-weeks. The assessment was conducted by three consultants over the course of eight calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Qingying Jie
✈ Engineer
qingying@zellic.io ↗

Kuilin Li
✈ Engineer
kuilin@zellic.io ↗

Chongyu Lv
✈ Engineer
chongyu@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 17, 2026 Start of primary review period

July 20, 2026 Kick-off call

July 28, 2026 End of primary review period

3. Detailed Findings

3.1. Counterfactual aToken poisoning blocks opening

Target	TydroPositionAccount		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The position account is deployed by the TydroPositionAccountFactory at a deterministic address. An attacker who knows the intended owner, Aave pool, asset pair, and position salt can transfer a small positive balance of the supply reserve's aToken to that address before its Aave collateral state is set.

```
// src/accounts/TydroPositionAccountFactory.sol:L70-L94
function createAccount(
    address owner,
    address pool,
    address supplyAsset,
    address borrowAsset,
    bytes32 userPositionSalt
) external returns (address account) {
    TydroPositionTypes.PositionIdentity memory positionIdentity =
        _validatedIdentity(owner, pool, supplyAsset, borrowAsset,
            userPositionSalt);
    bytes32 salt = _accountSalt(positionIdentity);
    bytes memory args = _cloneArgs(positionIdentity);

    account = Clones.predictDeterministicAddressWithImmutableArgs(
        accountImplementation, args, salt, address(this)
    );

    if (account.code.length != 0) return account;

    account
    = Clones.cloneDeterministicWithImmutableArgs(accountImplementation, args,
        salt);
    isAccount[account] = true;

    emit AccountCreated(
        owner, account, pool, supplyAsset, borrowAsset, userPositionSalt,
        implementationVersion
    );
}
```

}

During `open()`, the position account requests a mode-2 Aave flash loan. Inside the authenticated callback, `_settleOpenLifecycle()` supplies the signed initial collateral together with the measured swap output, but it does not explicitly enable the supply reserve as collateral before the callback returns, and Aave converts the flash-loan principal into variable debt.

```
// src/accounts/TydroPositionAccount.sol:L618-L642
function _settleOpenLifecycle(
    TydroPositionTypes.OpenIntent memory intent,
    uint256 flashLoanPremium,
    uint256 measuredCollateralAmount
) internal {
    address pool_ = pool();
    address supplyAsset_ = supplyAsset();
    if (intent.eModeCategory != 0) {
        uint256 currentEModeCategory
        = IPool(pool_).getUserEMode(address(this));
        if (currentEModeCategory != 0 && currentEModeCategory !=
            uint256(intent.eModeCategory)) revert InvalidLifecycleIntent();
        if (currentEModeCategory == 0) {
            IPool(pool_).setUserEMode(intent.eModeCategory);
        }
    }

    uint256 collateralAmountToSupply = intent.initialCollateralAmount
    + measuredCollateralAmount;
    supplyAsset_.forceApprove(pool_, collateralAmountToSupply);
    IPool(pool_).supply(supplyAsset_, collateralAmountToSupply, address(this),
    REFERRAL_CODE);
    supplyAsset_.forceApprove(pool_, 0);

    // Mode-2 flash loan: after this callback returns the pool converts the
    // flash-loaned
    // amount into variable debt on this account (no pull, no premium), so
    // there is nothing
    // to borrow or approve here. The signed flashLoanAmount IS the opened
    // debt.
    if (flashLoanPremium != 0)
        revert InvalidFlashLoanPremium(flashLoanPremium);
}
```

Aave automatically enables a supplied reserve as collateral only when the aToken mint reports that this is the recipient's first supply. A preexisting donated aToken balance makes `isFirstSupply` false, so the automatic collateral-enablement branch is skipped.

```
// aave-v3-origin/src/contracts/protocol/libraries/logic/SupplyLogic.sol:L72-L81
function executeSupply(
    mapping(address => DataTypes.ReserveData) storage reservesData,
    mapping(uint8 => DataTypes.EModeCategory) storage eModeCategories,
    DataTypes.UserConfigurationMap storage userConfig,
    DataTypes.ExecuteSupplyParams memory params
) external {
    // [...]

    // As aToken.mint rounds down the minted shares, we ensure an equivalent of
    // <= params.amount shares is minted.
    bool isFirstSupply = IAToken(reserveCache.aTokenAddress).mint(
        params.user,
        params.onBehalfOf,
        scaledAmount,
        reserveCache.nextLiquidityIndex
    );

    if (isFirstSupply) {
        if (
            ValidationLogic.validateUseAsCollateral(
                reservesData,
                eModeCategories,
                params.asset,
                params.supplierEModeCategory
            )
        ) {
            userConfig.setUsingAsCollateral(reserve.id, params.asset,
                params.onBehalfOf, true);
        }
    }
    // [...]
}
```

Consequently, the account can hold the expected aToken balance while Aave's separate collateral-configuration bit remains unset. When the mode-2 flash loan attempts to create variable debt after the callback, Aave validates the account as having no enabled collateral and reverts the opening transaction. Transferring the poisoned aTokens out through `claimATokens()` allows the owner to retry while no debt exists, but an attacker can donate another aToken balance before the retry.

Impact

Any user who can determine a future or unused position-account address can donate a small amount of the corresponding supply aToken and repeatedly prevent that account from completing its first opening life cycle. Each attempted `open()` reverts atomically, so the attacker neither steals collateral nor creates unauthorized debt. The victim, however, cannot establish the intended leveraged position when the price is favorable, pays for failed transactions when submitting them directly, and must diagnose the cause, claim the donated aTokens, and retry. The attack remains possible whenever the account has a positive aToken balance but the corresponding Aave collateral bit has not been enabled.

Recommendations

After supplying the collateral during `_settleOpenLifecycle()`, explicitly enable the configured supply reserve as collateral before returning from the flash-loan callback. The operation should be performed by the position account itself so that Aave updates the correct user's configuration, and any failure to enable the reserve should revert the life cycle before debt is created.

```
IPool(pool_).supply(supplyAsset_, collateralAmountToSupply, address(this),  
    REFERRAL_CODE);  
IPool(pool_).setUserUseReserveAsCollateral(supplyAsset_, true);
```

Retain the explicit collateral-enablement call even when the account appears to be receiving its first supply, because collateral safety should not depend on the account's prior aToken balance history. Add regression coverage for both a predeployment donation and a donation to a deployed but unused account.

Remediation

This issue has been acknowledged by Tydro, and a fix was implemented in commit [b4533004](#).

3.2. Failed open () calls can strand prefunded raw collateral

Target	TydroPositionAccount		
Category	Completeness	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The `OpenIntent.initialCollateralAmount` represents raw `supplyAsset` that is already held by the position account and that will be supplied to Aave together with the output of the opening swap. This permits an integration to transfer raw collateral to the account in one transaction and submit `open ( )` in a later transaction.

```
// src/types/TydroPositionTypes.sol:L100
struct OpenIntent {
    IntentCommon common;
    ConversionLeg conversion;
    address debtToken;
    uint256 initialCollateralAmount;
    uint8 eModeCategory;
    uint256 flashLoanAmount;
    uint256 minSuppliedTokenAmount;
    uint256 maxBorrowedTokenAmount;
    uint256 minHealthFactor;
}
```

The preheld amount is consumed only if the call to the function `open` succeeds – that is, this amount of tokens is supplied to Aave.

```
// src/accounts/TydroPositionAccount.sol:L618-L642
function _settleOpenLifecycle(
    TydroPositionTypes.OpenIntent memory intent,
    uint256 flashLoanPremium,
    uint256 measuredCollateralAmount
) internal {
    // [...]

    uint256 collateralAmountToSupply = intent.initialCollateralAmount +
        measuredCollateralAmount;
    supplyAsset_.forceApprove(pool_, collateralAmountToSupply);
}
```

```
IPool(pool_).supply(supplyAsset_, collateralAmountToSupply, address(this)
, REFERRAL_CODE);
supplyAsset_.forceApprove(pool_, 0);

// Mode-2 flash loan: after this callback returns the pool converts the
flash-loaned
// amount into variable debt on this account (no pull, no premium), so
there is nothing
// to borrow or approve here. The signed flashLoanAmount IS the opened
debt.
if (flashLoanPremium != 0)
    revert InvalidFlashLoanPremium(flashLoanPremium);
}
```

If the call to `open()` reverts because execution has been disabled, route configuration has changed, swap output is insufficient, or Aave rejects the life cycle, the revert cannot undo the already finalized transfer from the earlier transaction. The raw `supplyAsset` therefore remains in the position account. The account exposes no direct owner-only function for returning an idle raw `supplyAsset` balance.

The function `claimATokens()` does not recover this balance because it transfers the Aave reserve's `aToken` rather than the raw underlying token.

```
// src/accounts/TydroPositionAccount.sol:L320-L332
function claimATokens() external nonReentrant returns (uint256 claimedAmount)
{
    address owner_ = owner();
    if (msg.sender != owner_) revert Unauthorized(msg.sender);

    address aToken = _validatedSupplyAToken();
    claimedAmount = IERC20(aToken).balanceOf(address(this));
    if (claimedAmount == 0) revert InvalidAmount();

    aToken.safeTransfer(owner_, claimedAmount);
}
```

Meanwhile, `close()` rejects an account with no variable debt before it can reach the function that transfers residual raw supply assets to the signed receiver.

```
// src/accounts/TydroPositionAccount.sol:L265-L266
uint256 currentDebt = IERC20(localIntent.debtToken).balanceOf(address(this));
if (currentDebt == 0) revert InvalidAmount();
```

This observation applies only when raw collateral is transferred directly to the position account in a separate, nonatomic transaction. It does not apply to the intended front-end flow in which

collateral is supplied through Aave on behalf of the position account, because that flow creates `aTokens` that the owner can recover through `claimATokens()` while no debt exists. It also does not apply when the raw transfer and `open()` are performed atomically, because a failed `open()` then reverts the transfer in the same transaction.

Impact

A position owner or third-party integration that separately prefunds a position account with raw collateral can temporarily lose access to that collateral if the subsequent opening life cycle fails. The tokens are neither burned nor stolen and may become usable if a later `open()` succeeds and supplies them to Aave, but the owner has no direct way to transfer them out while opening remains unavailable. If the market remains disabled or no valid opening life cycle becomes available, the raw balance can remain stranded indefinitely.

Recommendations

Consider adding a narrowly scoped owner-only recovery function for idle balances of the account's immutable `supplyAsset` and `borrowAsset`. The function should transfer only to the immutable owner, remain protected against reentrancy, emit a recovery event, and avoid arbitrary calls or generalized token-execution authority.

If direct raw prefunding is intentionally unsupported, consider rejecting nonzero `initialCollateralAmount` values and documenting that integrations must use Aave `supply(..., onBehalfOf = positionAccount, ...)` or an atomic transfer-and-open transaction. This would align the contract surface with the intended front-end behavior and prevent integrations from relying on an unavailable recovery path.

Remediation

This issue has been acknowledged by Tydro, and a fix was implemented in commit [5516ad12](#).

3.3. SafeTransferLib treats calls to no-code addresses as successful ERC-20 calls

Target	SafeTransferLib		
Category	Integration Risks	Severity	Informational
Likelihood	Low	Impact	Informational

Description

SafeTransferLib uses low-level calls for transfer, transferFrom, and approve then treats a successful low-level call with either empty return data or a decoded true value as success. The library does not verify that the target address contains contract code, so a call to an externally owned account or any other address with no code will return `success = true` and empty return data. Under that behavior, the library reports a successful ERC-20 operation even though no token logic executed and no allowance or balance changed.

```
// src/libraries/SafeTransferLib.sol:L26-L44
function forceApprove(address token, address spender, uint256 amount)
    internal {
    bytes memory callData = abi.encodeCall(IERC20.approve, (spender, amount));
    (bool success, bytes memory data) = token.call(callData);

    if (!_didSucceed(success, data)) return;

    _callOptionalReturn(
        token, abi.encodeCall(IERC20.approve, (spender, 0)),
        IERC20.approve.selector
    );
    _callOptionalReturn(token, callData, IERC20.approve.selector);
}

function _callOptionalReturn(address token, bytes memory callData,
    bytes4 selector) private {
    (bool success, bytes memory data) = token.call(callData);
    if (!_didSucceed(success, data)) {
        revert TokenTransferFailed(token, selector);
    }
}

function _didSucceed(bool success, bytes memory data)
    private pure returns (bool) {
    return success && (data.length == 0 || abi.decode(data, (bool)));
}
```

Impact

In the codebase, many important call sites perform follow-up balance checks or higher-level configuration validation, which limits the direct security impact of this behavior. Even so, the library itself does not fail closed. A future integration that relies on SafeTransferLib as its only token validity check could silently accept a misconfigured token address and continue operating under the false assumption that the transfer or approval succeeded.

Recommendations

Consider checking that the target address contains code in the function `_didSucceed`.

```
function _didSucceed(bool success, bytes memory data) private pure returns (
    bool) {
    function _didSucceed(address token, bool success, bytes memory data)
        private view returns (bool) {
        return success && (data.length == 0 || abi.decode(data, (bool)));
        return success && ((data.length == 0 && token.code.length > 0) || abi.
            decode(data, (bool)));
    }
}
```

Remediation

This issue has been acknowledged by Tydro, and a fix was implemented in commit [7af694dc](#).

3.4. EMode transition checks are stricter than the underlying Aave rules

Target	TydroPositionAccount		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The contract TydroPositionAccount enforces a narrower eMode transition policy than the Aave Pool actually requires. In Aave, a user may enter, exit, or change eMode while positions are open as long as the resulting borrowed and collateral assets remain valid under the target category. By contrast, the account implementation assumes that changing eMode is only acceptable from a zero-debt state or that a nonzero eMode can only be reused exactly as is during a position open.

```
// src/accounts/TydroPositionAccount.sol:L298-L314
function setEModeCategory(uint8 categoryId) external nonReentrant {
    // [...]
    if (reserveDebtToken != address(0)) {
        uint256 currentDebt
        = IERC20(reserveDebtToken).balanceOf(address(this));
        if (currentDebt != 0) {
            revert EModeChangeRequiresZeroDebt(reserveDebtToken, currentDebt);
        }
    }

    IPool(pool_).setUserEMode(categoryId);
}
```

```
// src/accounts/TydroPositionAccount.sol:L826-L844
function _validateOpenEModeTransition(TydroPositionTypes.OpenIntent memory
    intent)
    internal
    view
{
    if (intent.eModeCategory == 0) return;

    uint256 currentEModeCategory = IPool(pool()).getUserEMode(address(this));
    if (currentEModeCategory != 0) {
        if (currentEModeCategory != uint256(intent.eModeCategory)) {
            revert InvalidLifecycleIntent();
        }
    }
}
```

```

        return;
    }

    uint256 currentDebt = IERC20(intent.debtToken).balanceOf(address(this));
    if (currentDebt != 0) {
        revert EModeChangeRequiresZeroDebt(intent.debtToken, currentDebt);
    }
}

```

The Aave validation logic is more permissive. It checks whether each outstanding borrowed asset remains borrowable in the target eMode and whether each enabled collateral asset remains valid in that state, rather than requiring the account to have no debt.

Impact

The codebase can reject valid Aave eMode transitions even when the resulting position would satisfy the underlying protocol's own safety checks.

The stricter local policy can block legitimate operator actions and life-cycle flows that would be acceptable to Aave. For example, an account with outstanding debt may be unable to migrate into a compatible eMode category without first fully deleveraging. This adds operational friction and complicates account management.

Recommendations

Consider removing the local prechecks and relying on the Aave Pool's own validation logic to enforce eMode transition rules.

Remediation

This issue has been acknowledged by Tydro, and a fix was implemented in commit [3fa8f25d](#).

3.5. Active flash-loan callback commitments remain live during callback execution

Target	TydroPositionAccount		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The flash-loan callback authentication scheme leaves the active callback commitment live for the entire duration of callback execution. During a position open or close, the account clears transient state, stores an `ACTIVE_*_CALLBACK_HASH_SLOT`, and then requests the flash loan.

```
// src/accounts/TydroPositionAccount.sol:L228-L235
function open(
    TydroPositionTypes.OpenIntent calldata intent,
    bytes calldata signature,
    bytes calldata routeData
) external nonReentrant returns (uint256 measuredAmountOut) {
    // [...]

    _clearOpenCallbackState();
    TydroTransientStorage.storeBytes32(ACTIVE_OPEN_CALLBACK_HASH_SLOT,
        callbackHash);
    _requestFlashLoan(localIntent.flashLoanAmount, encodedContext,
        VARIABLE_INTEREST_MODE);

    bytes32 pendingCallbackHash =
        TydroTransientStorage.loadBytes32(PENDING_OPEN_CALLBACK_HASH_SLOT);
    measuredAmountOut =
        TydroTransientStorage.loadUint256(PENDING_OPEN_MEASURED_AMOUNT_OUT_SLOT);
    _clearOpenCallbackState();

    // [...]
}
```

Inside the function `executeOperation`, the account checks that the incoming params hash matches the active slot, but it does not clear that slot before executing the swap and settlement logic. Instead, the active hash remains present until the callback body finishes and the code copies it into the pending slot that the outer `open()` or `close()` call later verifies.

```
// src/accounts/TydroPositionAccount.sol:L344-L396
function executeOperation(
    address[] calldata assets,
    uint256[] calldata amounts,
    uint256[] calldata premiums,
    address initiator,
    bytes calldata params
) external returns (bool) {
    if (msg.sender != pool()) revert Unauthorized(msg.sender);
    if (initiator != address(this)) {
        revert InvalidFlashLoanInitiator(address(this), initiator);
    }
    if (assets.length != 1 || amounts.length != 1 || premiums.length != 1) {
        revert InvalidFlashLoanArrayLength(assets.length, amounts.length,
        premiums.length);
    }
    address borrowAsset_ = borrowAsset();
    if (assets[0] != borrowAsset_) revert InvalidFlashLoanAsset(borrowAsset_,
    assets[0]);

    bytes32 callbackHash = keccak256(params);
    bytes32 activeOpenCallbackHash =
        TydroTransientStorage.loadBytes32(ACTIVE_OPEN_CALLBACK_HASH_SLOT);
    if (activeOpenCallbackHash != bytes32(0)) {
        if (callbackHash != activeOpenCallbackHash)
            revert InvalidFlashLoanCallbackState();
        _executeOpenFlashLoanCallback(amounts[0], premiums[0], params);
        return true;
    }
    bytes32 activeCloseCallbackHash =
        TydroTransientStorage.loadBytes32(ACTIVE_CLOSE_CALLBACK_HASH_SLOT);
    if (activeCloseCallbackHash != bytes32(0)) {
        if (callbackHash != activeCloseCallbackHash)
            revert InvalidFlashLoanCallbackState();
        _executeCloseFlashLoanCallback(amounts[0], premiums[0], params);
        return true;
    }

    revert InvalidFlashLoanCallbackState();
}

function _executeOpenFlashLoanCallback(uint256 amount, uint256 premium,
    bytes calldata params)
    internal
{
    OpenCallbackContext memory context = abi.decode(params,
```

```
(OpenCallbackContext));
if (amount != context.intent.flashLoanAmount) {
    revert InvalidFlashLoanAmount(context.intent.flashLoanAmount, amount);
}

uint256 measuredAmountOut =
    _executeOpenConversionLeg(context.intent.conversion,
context.routeData);
_settleOpenLifecycle(context.intent, premium, measuredAmountOut);

bytes32 activeOpenCallbackHash =
    TydroTransientStorage.loadBytes32(ACTIVE_OPEN_CALLBACK_HASH_SLOT);
TydroTransientStorage.storeBytes32(PENDING_OPEN_CALLBACK_HASH_SLOT,
activeOpenCallbackHash);
TydroTransientStorage.storeUint256(PENDING_OPEN_MEASURED_AMOUNT_OUT_SLOT,
measuredAmountOut);
}
```

Impact

This is not directly exploitable because the callback must originate from the immutable Pool, the initiator must be the account itself, and the codebase already treats Aave as a trusted dependency. However, from a defense-in-depth perspective, the callback hash is being used as an authentication guard precisely because the account does not want to trust arbitrary callback invocations. Leaving that guard reusable throughout callback execution weakens the fail-closed posture of the mechanism.

If the configured Pool were ever to invoke the function `executeOperation` more than once for the same active commitment, or to otherwise violate the account's assumptions about callback uniqueness, the life-cycle body could execute more than once before the outer call clears transient state. That could duplicate swaps, duplicate collateral supply or debt-settlement side effects, and overwrite the measured amount that the outer life cycle later consumes.

Recommendations

Consider invalidating the active callback slot immediately after the function `executeOperation` loads it and before any external calls. The callback can then record completion into the pending slot after execution succeeds.

The same pattern should be applied symmetrically to both open and close callbacks.

Remediation

This issue has been acknowledged by Tydro, and a fix was implemented in commit [a5dda5f1](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Discussion of registry pool configurability

The registry stores a pool address in each route registration, but the surrounding architecture makes that field much less flexible than it may initially appear. The market key is derived only from the ordered (supplyToken, borrowToken) pair.

```
// src/libraries/ValidationLib.sol:L21-L33
function computeMarketId(address supplyToken, address borrowToken)
    internal
    pure
    returns (bytes32 marketId)
{
    // Packed market id: keccak256(abi.encodePacked(supplyToken,
    borrowToken)).
    assembly ("memory-safe") {
        let ptr := mload(0x40)
        mstore(ptr, shl(96, supplyToken))
        mstore(add(ptr, 20), shl(96, borrowToken))
        marketId := keccak256(ptr, 40)
    }
}
```

The factory embeds the pool address into each account's identity and deterministic address.

```
// src/accounts/TydroPositionAccountFactory.sol:L70-L94
function createAccount(
    address owner,
    address pool,
    address supplyAsset,
    address borrowAsset,
    bytes32 userPositionSalt
) external returns (address account) {
    TydroPositionTypes.PositionIdentity memory positionIdentity =
        _validatedIdentity(owner, pool, supplyAsset, borrowAsset,
        userPositionSalt);
    bytes32 salt = _accountSalt(positionIdentity);
    bytes memory args = _cloneArgs(positionIdentity);

    account = Clones.predictDeterministicAddressWithImmutableArgs(
```

```

        accountImplementation, args, salt, address(this)
    );

    if (account.code.length != 0) return account;

    account
    = Clones.cloneDeterministicWithImmutableArgs(accountImplementation, args,
    salt);
    isAccount[account] = true;

    emit AccountCreated(
        owner, account, pool, supplyAsset, borrowAsset, userPositionSalt,
    implementationVersion
    );
}

```

At the same time, runtime validation requires the position account's immutable pool() to match the registration's stored pool exactly.

```

// src/registry/TydroMarketRegistry.sol:L444-L475
function validateSwapExecution(TydroPositionTypes.SwapExecutionCheck calldata
    check)
    external
    view
    override
{
    bytes32 marketId = ValidationLib.computeMarketId(check.supplyAsset,
    check.borrowAsset);

    _validateLifecycleDirection(check);

    TydroTypes.RouteRegistration storage registration
    = _requireRegisteredRoute(marketId);
    // [...]
    if (registration.pool != check.pool) {
        revert PoolMismatch(registration.pool, check.pool);
    }
    // [...]
}

```

Taken together, these choices make the stored registration.pool a registration-time namespace decision. A single registry can only hold one approved Pool for a given ordered asset pair, because the function registerRoute is one-shot on that pair-derived market ID and there is no Pool update function. Even if a Pool update function were added later, existing accounts would still remain immutably tied to the original Pool through their clone arguments and address derivation.

It is internally consistent if the intended architecture is that each registry and factory pair serves one canonical Pool universe for each asset pair. However, if the protocol may eventually want the same (supplyAsset, borrowAsset) pair to coexist across multiple Aave Pools, then the present model will force that expansion to happen through separate registries and factories or through a breaking key-space change that includes Pool in the market identity. In that future, making Pool part of the market key would more accurately reflect the true execution boundary than treating it as auxiliary registration metadata.

4.2. The exact-input life cycle leaves `ConversionLeg.maxAmountIn` redundant

The position account life cycle only supports exact-input swaps. The signed `ConversionLeg` still contains `maxAmountIn`, and the function `validateConversionLeg` checks that `amountIn` does not exceed it, but the route and account execution paths both require the conversion to consume exactly `amountIn`.

This makes `maxAmountIn` functionally redundant. Consider removing it from the `ConversionLeg` struct and any associated validation logic. This would simplify the data model and reduce potential confusion for developers interacting with the API.

5. Threat Model

This section provides a threat model for the security-critical functions in the audited contracts. Each function model describes externally controllable inputs, enforced constraints, security impact, intended execution paths, and negative behaviors covered by the test suite.

A checked item means that at least one test or invariant reaches the named behavior and asserts its outcome. An unchecked item means that no behavior-specific assertion was identified at the audited commit; incidental line coverage through a shared modifier, helper, or different external entry point is not treated as coverage for that item.

Simple getters, constructors, interfaces, constants, errors, and internal encoding or storage helpers are not modeled separately. Their relevant assumptions and validation behavior are incorporated into the models of the life-cycle, authority, policy, and swap entry points that consume them. The absence of a function-level model does not by itself imply that a function is safe.

5.1. Module: TydroGuardian

Function: `acceptOwnership()`

This function completes the Guardian's two-step ownership transfer, clears the pending owner, and makes the caller the sole authority for immediate defensive registry actions.

Inputs

This function has no inputs.

Branches and code coverage

Intended branches

- ☒ The pending owner should accept ownership and gain the ability to disable route execution.
- ☒ The former owner should lose immediate defensive authority after acceptance.

Negative behavior

- ☒ Revert when the caller is not the currently nominated pending owner.

Function: `disableRouteExecution(TydroMarketRegistry registry, bytes32 marketId)`

This owner-only wrapper requests immediate disablement of a registered market through the Guardian-only registry boundary. The Guardian contract itself cannot enable a market or exercise position-account lending authority.

Inputs

- `registry`
 - **Control:** Full control.
 - **Constraints:** The caller must own this Guardian, and the target registry must recognize this contract as its configured Guardian for the call to succeed.
 - **Impact:** Selects the registry in which the defensive action is attempted.
- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify a market registered in the selected registry.
 - **Impact:** Selects the market whose official life-cycle execution is disabled and whose activation state becomes stale.

Branches and code coverage

Intended branches

- ☒ The Guardian owner should immediately disable an enabled market through the configured registry.
- ☒ The Guardian wrapper should invalidate a queued first activation even when the market is already disabled.
- ☐ Repeated calls through this wrapper should remain valid and advance the registry policy epoch on every call.
- ☒ Guardian ownership rotation should transfer this immediate power to the new owner.

Negative behavior

- ☒ Revert when the caller is not the Guardian owner.
- ☒ The Guardian owner should remain unable to call the registry's owner-only enablement function.
- ☒ Revert when the selected registry no longer authorizes this Guardian after Guardian rotation.
- ☐ Revert when the market is unregistered.

Function: `tightenRouteRiskCaps(TydroMarketRegistry registry, bytes32 marketId, TydroTypes.RiskCaps riskCaps)`

This owner-only wrapper requests an immediate non-loosening replacement of a market's complete health-factor and slippage policy through the Guardian-only registry boundary.

Inputs

- `registry`
 - **Control:** Full control.
 - **Constraints:** The caller must own this Guardian, and the target registry must recognize this contract as its configured Guardian.
 - **Impact:** Selects the registry whose defensive risk-control entry point is invoked.
- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify a market registered in the selected registry.
 - **Impact:** Selects the market whose official life-cycle risk policy is tightened.
- `riskCaps`
 - **Control:** Full control.
 - **Constraints:** Must satisfy all absolute registry bounds and may only raise the existing minimum health factor, lower the existing maximum health factor, or reduce either slippage cap. Equal values are permitted.
 - **Impact:** Defines the complete equal-or-stricter policy written to the market and invalidates pending activation state.

Branches and code coverage

Intended branches

- ☒ The Guardian owner should reach the same risk-tightening path through this wrapper while the market is disabled.
- ☐ The wrapper should store changed values for all four risk-cap dimensions while the market is enabled.
- ☒ Equal-value caps should be accepted to invalidate queued activation without changing policy values.
- ☒ Timelocked governance should remain able to loosen a cap later through the owner-only registry path.

Negative behavior

- ☒ Revert when any proposed policy dimension loosens the current policy.
- ☒ Revert when a nominally tighter value violates an absolute bound, such as zero slippage.
- ☐ Revert directly at this wrapper when the caller is not the Guardian owner.

Function: `transferOwnership(address newOwner)`

This owner-only function nominates a successor for the Guardian's immediate defensive authority while leaving that authority with the current owner until the nominee accepts.

Inputs

- `newOwner`
 - **Control:** Full control.
 - **Constraints:** The caller must be the current Guardian owner, and the nominee must not be the zero address.
 - **Impact:** Selects the only address allowed to complete the handoff and gain immediate route-disablement and risk-tightening authority.

Branches and code coverage

Intended branches

- ☒ The current owner should nominate a pending owner without immediately transferring defensive power.
- ☒ Acceptance by the nominee should later move immediate defensive power to that address.

Negative behavior

- ☒ Revert when the nominee is the zero address.
- ☐ Revert when the caller is not the current Guardian owner.

5.2. Module: TydroMarketRegistry

Function: `acceptOwnership()`

This function completes the registry's two-step ownership transfer, grants all owner-only policy authority to the nominated address, and clears the pending owner.

Inputs

This function has no inputs.

Branches and code coverage

Intended branches

- ☑ The pending owner should accept ownership and clear the pending-owner slot.
- ☑ The former owner should lose policy privileges after acceptance while the new owner gains them.

Negative behavior

- ☑ Revert when the caller is not the currently nominated pending owner.

Function: `enableRoute(bytes32 marketId, bytes32 expectedStateHash)`

This owner-only function enables official life-cycle execution only when the supplied commitment exactly matches the current chain, registry, complete market registration, registry policy epoch, and route-local configuration hash and epoch. Successful activation consumes the snapshot by advancing the policy epoch.

Inputs

- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify a registered market whose execution is currently disabled.
 - **Impact:** Selects the market whose approved route becomes available to official position-account open and close life cycles.
- `expectedStateHash`
 - **Control:** Full control.
 - **Constraints:** Must exactly equal the current value committed by `routeStateHash`, including the current route-local config and both monotonic epochs.
 - **Impact:** Binds delayed governance execution to the exact market state that was reviewed when activation was scheduled.

Branches and code coverage

Intended branches

- ☑ The owner should enable a disabled route with its exact current state hash.
- ☑ Successful activation should advance the policy epoch, emit the expected events, and make the consumed snapshot unusable again.
- ☑ A delayed activation should execute through the governance timelock after the configured delay.

Negative behavior

- ☒ Revert when the caller is not the registry owner.
- ☒ Revert for an unregistered or already enabled market.
- ☒ Revert after an intervening Guardian disable, risk-cap write, registry config update, route replacement, or route-local path write changes the committed state.
- ☒ Revert when route-local and registry configuration hashes drift, even if the caller supplies the newest state hash.
- ☒ Revert when state is changed away and back because monotonic policy and route-config epochs prevent snapshot revival.
- ☒ Revert when the same queued activation is submitted more than once.

Function: guardianDisableRouteExecution(bytes32 marketId)

This Guardian-only function immediately disables official life-cycle execution for a registered market. It advances the policy epoch even when the market is already disabled so the Guardian can invalidate queued first activation or reactivation without gaining any enablement authority.

Inputs

- marketId
 - **Control:** Full control.
 - **Constraints:** The caller must equal the configured Guardian, and the market must be registered.
 - **Impact:** Selects the market whose official open and close execution is stopped and whose pending activation commitments become stale.

Branches and code coverage

Intended branches

- ☒ The configured Guardian should immediately disable an enabled market.
- ☒ Repeated disablement while already disabled should remain valid and advance the policy epoch each time.
- ☒ Guardian disablement should invalidate a queued first activation and a queued reactivation.
- ☒ Policy invariants should prove that Guardian actions can never enable execution.

Negative behavior

- ☒ Revert when the caller is not the configured Guardian.
- ☐ Revert when the market is not registered.

Function: `guardianTightenRouteRiskCaps(bytes32 marketId, TydroTypes.RiskCaps riskCaps)`

This Guardian-only function immediately applies a complete non-loosening risk policy. It can raise the minimum target health factor, lower the maximum target health factor, or reduce either slippage cap, and equal-value writes remain available to invalidate queued activation.

Inputs

- `marketId`
 - **Control:** Full control.
 - **Constraints:** The caller must equal the configured Guardian, and the market must be registered.
 - **Impact:** Selects the market whose life-cycle policy is restricted or whose activation commitment is invalidated.
- `riskCaps`
 - **Control:** Full control.
 - **Constraints:** Must satisfy protocol-wide health-factor and slippage bounds and may not lower the existing minimum health factor, raise the existing maximum health factor, or raise either existing slippage cap.
 - **Impact:** Replaces the complete market risk envelope with equal or stricter limits and advances the policy epoch.

Branches and code coverage

Intended branches

- ☒ The Guardian should tighten and store every health-factor and slippage dimension while execution is enabled.
- ☒ Guardian risk-cap writes should also succeed while execution is disabled and should advance the policy epoch.
- ☒ The Guardian should reapply equal caps to invalidate pending activation without changing values.
- ☒ Directionality tests should distinguish Guardian tightening from timelocked owner loosening.

Negative behavior

- ☐ Revert through `guardianTightenRouteRiskCaps` when the caller is not the configured Guardian.
- ☐ Revert when the market is unregistered.
- ☒ Revert when the proposed minimum health factor decreases, maximum health factor increases, or either slippage cap increases.

- ☒ Revert when the replacement violates the protocol's absolute health-factor or slippage bounds.

Function: `registerRoute(TydraTypes.RouteRegistration registration)`

This owner-only function creates the one-time disabled policy record for an ordered supply-token and borrow-token market. It binds the market to one deployed Aave Pool, one registry-bound route, validated health-factor and slippage caps, and a route-local configuration commitment before any life cycle can be enabled.

Inputs

- `registration`
 - **Control:** Full control.
 - **Constraints:** The caller must be the owner; supply and borrow tokens must be nonzero and distinct; the route and Pool must contain code; the route must report this registry and the same nonzero local configuration hash; execution must initially be disabled; risk-cap health factors must lie within protocol hard limits in noninverted order; both slippage caps must be between 1 and 9,999 basis points; and the derived market must not already be registered.
 - **Impact:** Establishes the immutable asset pair and Pool plus the initially approved route, route configuration commitment, life-cycle risk policy, and first registry policy epoch.

Branches and code coverage

Intended branches

- ☒ The owner should register a valid disabled market and store every route, Pool, asset, risk-cap, and configuration field.
- ☒ Registration should initialize the policy epoch to one and emit registration and epoch events.
- ☒ A delayed timelock batch should configure the route and register the market in the intended production order.

Negative behavior

- ☒ Revert when the caller is not the owner or the registration is already enabled.
- ☒ Revert through `registerRoute` for a zero or codeless route or a zero or codeless Pool.
- ☒ The shared registration validator used by `registerRoute` should reject zero or identical tokens, a zero configuration hash, and invalid risk caps.
- ☒ Revert when the route is bound to another registry, its config-state call fails, or its local configuration hash differs from the registration.

- ☒ Revert when the directional market has already been registered.

Function: `setGuardian(address newGuardian)`

This owner-only function replaces the address authorized to invoke the registry's immediate disablement and non-loosening risk-cap controls.

Inputs

- `newGuardian`
 - **Control:** Full control.
 - **Constraints:** The caller must be the registry owner, and the address must be nonzero. The contract does not require deployed code or verify Guardian capabilities.
 - **Impact:** Transfers immediate defensive authority over every registered market to the selected address.

Branches and code coverage

Intended branches

- ☒ Timelocked governance should replace the registry Guardian and transfer immediate defensive power to the new Guardian contract.

Negative behavior

- ☒ Revert when the caller is not the registry owner.
- ☒ Revert when the new Guardian is the zero address.
- ☐ Deployment or governance tests should reject or explicitly document an EOA or incompatible contract as the Guardian target.

Function: `setRouteEnabled(bytes32 marketId, bool enabled)`

This compatibility entry point allows the registry owner to disable a registered market and invalidate previously prepared activation calldata. It deliberately cannot enable execution because activation must use the checked state-commitment path.

Inputs

- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify an existing directional market registration.

- **Impact:** Selects the market whose official account life-cycle execution is disabled and whose policy epoch advances.
- enabled
 - **Control:** Full control.
 - **Constraints:** Must be false — true is accepted only by enableRoute with an exact state hash.
 - **Impact:** Expresses the disable-only operation and cannot grant execution authority.

Branches and code coverage

Intended branches

- ☒ The owner should disable a registered market.
- ☒ Every disable write should advance the market policy epoch and invalidate stale activation state.

Negative behavior

- ☒ Revert when the caller is not the owner.
- ☒ Revert when the market is not registered.
- ☒ Revert when enabled is true.

Function: `setRouteRiskCaps(bytes32 marketId, TydroTypes.RiskCaps riskCaps)`

This owner-only function replaces the complete health-factor and oracle-relative slippage policy for a disabled market. Because the owner is intended to be a timelock, it may tighten or loosen caps within protocol-wide hard bounds.

Inputs

- marketId
 - **Control:** Full control.
 - **Constraints:** Must identify a registered market whose execution is disabled.
 - **Impact:** Selects the market whose signed life cycle and swap-risk envelope changes.
- riskCaps
 - **Control:** Full control.
 - **Constraints:** Minimum and maximum target health factors must lie within protocol hard bounds and may not be inverted; open and close slippage caps must each be between 1 and 9,999 basis points.

- **Impact:** Defines the accepted signed health-factor band and maximum oracle-relative slippage for future opening and closing life cycles.

Branches and code coverage

Intended branches

- ☒ The owner should replace all risk-cap fields for a disabled market.
- ☒ A valid write should advance the policy epoch and invalidate a previously prepared activation.

Negative behavior

- ☒ Revert when the caller is not the owner, the market is unregistered, or execution is enabled.
- ☒ The shared risk-cap validator used by `setRouteRiskCaps` should reject health factors below or above hard bounds, an inverted health-factor range, zero slippage, and slippage at or above 10,000 basis points.

Function: `transferOwnership(address newOwner)`

This owner-only function nominates a successor for the registry's delayed-governance authority while leaving all policy mutation privileges with the current owner until acceptance.

Inputs

- `newOwner`
 - **Control:** Full control.
 - **Constraints:** The caller must be the current owner, and the nominee must not be the zero address.
 - **Impact:** Selects the only address permitted to complete the second step and acquire authority over registration, activation, route rotation, risk caps, and Guardian replacement.

Branches and code coverage

Intended branches

- ☒ The `pendingOwner` is set to the `newOwner` address.
- ☒ The current owner remains unchanged.

Negative behavior

- ☒ Revert when the caller is not the current owner.

- ☒ Revert when the nominee is the zero address.

Function: `updateConfigHash(bytes32 marketId, bytes32 expectedOldConfigHash, bytes32 newConfigHash)`

This owner-only function replaces the registry's commitment to a disabled route's local swap-path configuration. It uses the expected old hash as a stale-write guard and is intended to execute in the same ordered governance batch as the corresponding route-local path rotation.

Inputs

- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify a registered market whose execution is disabled.
 - **Impact:** Selects the market whose approved route configuration commitment changes.
- `expectedOldConfigHash`
 - **Control:** Full control.
 - **Constraints:** Must equal the registration's current configuration hash.
 - **Impact:** Prevents stale governance calldata from overwriting a configuration that has changed since scheduling.
- `newConfigHash`
 - **Control:** Full control.
 - **Constraints:** Must be nonzero. This function does not itself require the route-local path to already report the new hash; checked activation later requires equality.
 - **Impact:** Becomes the registry-side commitment that future official route execution and activation must match.

Branches and code coverage

Intended branches

- ☒ The owner should update the stored configuration commitment for a disabled market.
- ☒ The production governance payload should order route-local rotation and registry hash update in one delayed batch.
- ☒ A successful update should advance the policy epoch and invalidate pending activation state.

Negative behavior

- ☒ Revert when the caller is not the owner, the market is unregistered, or execution is

enabled.

- ☒ Revert when the new hash is zero or the expected old hash is stale.
- ☒ Checked activation should continue to fail until the route-local hash and registry commitment agree.

Function: `updateRoute(bytes32 marketId, address expectedOldRoute, address newRoute)`

This owner-only function replaces the route implementation approved for a disabled market. It guards against stale writes and requires the replacement to contain code, bind immutably to this registry, and report a local configuration hash equal to the existing registry commitment.

Inputs

- `marketId`
 - **Control:** Full control.
 - **Constraints:** Must identify a registered market whose execution is disabled.
 - **Impact:** Selects the market whose official life-cycle route implementation changes.
- `expectedOldRoute`
 - **Control:** Full control.
 - **Constraints:** Must equal the currently approved route.
 - **Impact:** Prevents stale replacement calldata from overwriting a more recent route update.
- `newRoute`
 - **Control:** Full control.
 - **Constraints:** Must be a deployed contract bound to this registry, and its local configuration hash for the market must equal the current registry commitment.
 - **Impact:** Becomes the only route accepted by the registry for future official open and close lifecycles.

Branches and code coverage

Intended branches

- ☒ The owner should replace the approved route while execution is disabled and advance the policy epoch.
- ☒ A delayed atomic replacement should update the route and configuration, leave execution disabled, invalidate prior activation, and require separate reactivation.

Negative behavior

- ☒ Revert for an unregistered or enabled market, a non-owner caller, or a stale expected old route.
- ☒ Revert when the replacement is bound to another registry or reports a local configuration hash different from the current registry commitment.
- ☐ Revert explicitly when the replacement route is the zero address or contains no deployed code.

Function: `validateSwapExecution(TydraPositionTypes.SwapExecutionCheck check)`

This view function validates an official position-account conversion against current market policy. It derives the directional market, verifies open or close direction, enabled status, approved route, Pool, configuration commitment, signed health-factor band, and route-to-registry binding, then enforces an action-specific minimum output derived from live Aave oracle prices and token decimals.

Inputs

- `check`
 - **Control:** Full control.
 - **Constraints:** The ordered asset pair must identify a registered and enabled market; the conversion direction must match `Open` or `Close`; the Pool, route, and configuration hash must match registration; the route must remain bound to this registry; the signed health-factor floor must lie within the registered band; oracle prices must be nonzero; and the signed minimum output must be no lower than oracle fair value after the action-specific slippage cap.
 - **Impact:** Determines whether an account may proceed to flash-loan and route execution under the current control-plane and oracle risk policy.

Branches and code coverage

Intended branches

- ☒ Valid opening and closing checks should pass for the registered Pool, route, configuration, health-factor band, and conversion direction.
- ☒ Opening should use the open slippage cap, and closing should use the close slippage cap.
- ☒ The registry oracle floor should track changes in live Aave oracle prices.
- ☒ The shared oracle fair-value calculation used by the registry should convert between differing token decimals.

- ☒ Policy invariants should track the registry's enabled state across owner and Guardian mutations.

Negative behavior

- ☒ Revert for incorrect open or close direction, an unregistered or disabled market, unapproved route, Pool mismatch, or configuration-hash drift.
- ☒ Revert when the signed health-factor floor lies below or above the registered band.
- ☒ Revert when the route no longer reports this registry or its registry-binding call fails.
- ☒ Revert when either Aave oracle price is zero or the signed minimum output lies below the calculated opening or closing risk floor.
- ☐ Explicitly cover token contracts whose `decimals()` call reverts or returns an extreme value that makes oracle normalization fail closed.

5.3. Module: TydroPositionAccount

Function: `claimATokens()`

This owner-only escape function transfers the position account's complete balance of the configured supply reserve's aToken to the immutable owner. Aave's aToken transfer validation independently prevents the transfer when the collateral is still required to support debt.

Inputs

This function has no inputs.

Branches and code coverage

Intended branches

- ☒ The owner should receive the complete configured supply aToken balance.
- ☒ The recipient should always be the immutable owner rather than the transaction submitter or a caller-selected address.
- ☒ After manual debt repayment, the owner should claim the complete aToken balance on the pinned Ink Aave deployment and withdraw the underlying directly from Aave.

Negative behavior

- ☒ Revert when the caller is not the immutable owner.
- ☒ Revert when the account has no configured supply aToken balance.
- ☒ Revert atomically on the live Aave pool when transferring the aTokens would leave outstanding debt undercollateralized.
- ☐ Revert when `claimATokens` itself observes a zero or codeless aToken address for the configured Aave reserve.

Function: `close(TydroPositionTypes.CloseIntent intent, bytes signature, bytes routeData)`

This function closes an Aave leveraged position under an owner-signed EIP-712 intent. It obtains the signed repayment principal through a mode-0 flash loan, repays variable debt, withdraws collateral, swaps the exact signed collateral input back to the borrow asset, reserves principal plus premium for Aave, forwards permitted residual assets, and verifies the remaining debt, collateral, and health factor.

Inputs

- `intent`
 - **Control:** Full control.
 - **Constraints:** The intent must bind this account and its immutable identity, current nonce, valid caller and deadline, correctly directed and approved conversion, matching Aave variable-debt token, nonzero repayment and withdrawal bounds, nonzero residual receiver, and a `maxFlashLoanRepayment` not below `maxRepayAmount`. Current debt must be nonzero and no greater than `maxRepayAmount`; the signed health-factor floor, route, configuration, and minimum output must satisfy current registry policy.
 - **Impact:** Defines the flash-loan principal and repayment ceiling, collateral amount exchanged, swap protections, permitted remaining position, and recipients of borrow-asset surplus and raw supply-asset residuals.
- `signature`
 - **Control:** Full control.
 - **Constraints:** Must be a valid ECDSA signature from the immutable EOA owner over the complete `CloseIntent`, this account's EIP-712 domain, and the current nonce.
 - **Impact:** Authorizes debt repayment, collateral withdrawal, conversion, and residual distribution under the exact signed close envelope.
- `routeData`
 - **Control:** Full control.
 - **Constraints:** Its hash must equal `intent.conversion.routeDataHash`, and the approved route must accept the encoded market, configuration hash, reverse direction, and slippage limit.
 - **Impact:** Selects the exact route-specific execution data for converting withdrawn collateral into the flash-loan repayment asset.

Branches and code coverage

Intended branches

- ☑ A valid close should execute the mode-0 flash loan, debt repayment, collateral withdrawal, reverse swap, flash repayment, and postconditions atomically.
- ☑ The signed repayment cap should tolerate debt-snapshot dust and should be used as the exact flash-loan principal.
- ☑ Borrow-asset surplus and raw supply-asset residuals should be transferred to the signed residual receiver.
- ☑ Closing should withdraw donated aToken dust and return the resulting supply-asset residual.
- ☑ A complete close should clear debt and collateral while retaining sufficient principal and premium for the Aave pull.
- ☑ The complete signed close life cycle should execute against the pinned Ink Aave deployment.

Negative behavior

- ☑ Revert for invalid authorization, incorrect conversion direction, route-data mismatch, incorrect debt token, zero debt, or debt above the signed repayment cap.
- ☑ Revert when the route underdelivers, overpulls, reports misleading output, attempts reentrancy, or tries to exercise Aave authority owned by the account.
- ☑ Revert when withdrawn collateral is below the signed minimum or remaining debt, collateral, or health factor violates its signed postcondition.
- ☑ Revert when principal plus premium exceeds `maxFlashLoanRepayment` or the account lacks enough borrow asset for repayment.
- ☑ Revert for an incorrect flash-loan amount, missing active callback, missing pending callback hash, or a tampered second callback.
- ☑ Tested late-stage close failures should revert atomically and preserve the original position and nonce.
- ☑ Revert when registry execution is disabled, with fork coverage demonstrating the separate manual Aave deleverage and aToken-claim escape path.

Function: `executeOperation(address[] assets, uint256[] amounts, uint256[] premiums, address initiator, bytes params)`

This Aave flash-loan callback services the single active open or close life cycle committed in transient storage. It authenticates the immutable Pool and account initiator, validates the single borrow asset and callback context, and then dispatches to the opening or closing settlement path.

Inputs

- `assets`
 - **Control:** Full control.
 - **Constraints:** The caller must be the immutable Aave Pool, the array must contain exactly one element, and that element must equal the account's

- immutable borrow asset.
- **Impact:** Identifies the asset received from Aave and used by the active life cycle.
- amounts
 - **Control:** Full control.
 - **Constraints:** The array must contain exactly one principal equal to the amount committed by the active open or close callback context.
 - **Impact:** Determines the amount swapped into collateral during open or used to repay debt during close.
- premiums
 - **Control:** Full control.
 - **Constraints:** The array must contain exactly one value. Open requires a zero mode-2 premium; close includes the premium in the bounded mode-0 repayment amount.
 - **Impact:** Determines the additional borrow-asset amount reserved for Aave during close and is rejected if unexpected during open.
- initiator
 - **Control:** Full control.
 - **Constraints:** Must equal the position account itself.
 - **Impact:** Prevents third parties from initiating a flash loan that exercises the account's life-cycle callback authority.
- params
 - **Control:** Full control.
 - **Constraints:** Its hash must equal the active transient callback commitment, and it must decode to the exact open or close intent, route data, and close principal committed before the flash loan.
 - **Impact:** Selects the active life cycle and carries all signed settlement instructions into the callback.

Branches and code coverage

Intended branches

- ☒ An authenticated mode-2 callback should execute the opening swap and Aave supply settlement.
- ☒ An authenticated mode-0 callback should repay debt, withdraw collateral, execute the closing swap, reserve repayment, and enforce postconditions.
- ☒ The callback should store the matching pending hash and measured output for verification after Aave returns control.

Negative behavior

- ☑ Revert when the caller is not the immutable Aave Pool or the initiator is not the account.
- ☑ Revert for mismatched array lengths, incorrect asset, incorrect principal, or nonzero opening premium.
- ☑ Revert when no life-cycle callback is active or when params does not match the active transient commitment.
- ☑ Revert when a second callback attempts to reuse or tamper with an active life-cycle context.
- ☑ Revert atomically when settlement open/close, route execution, repayment, or postcondition validation fails.

Function: `open(TydroPositionTypes.OpenIntent intent, bytes signature, bytes routeData)`

This function opens or increases an Aave leveraged position under an owner-signed EIP-712 intent. It validates the account identity and current registry policy, requests a mode-2 flash loan of the borrow asset, swaps the exact principal into collateral through the approved route, supplies the resulting collateral to Aave, and verifies the final aToken balance, debt, and health factor.

Inputs

- `intent`
 - **Control:** Full control.
 - **Constraints:** The intent must identify this account, its immutable owner, Aave Pool, supply asset, and borrow asset; name the current nonce and an unexpired deadline; authorize the caller or allow any relayer; specify a nonzero and correctly directed conversion whose exact input equals the mode-2 flash-loan amount; name the Aave variable-debt token for the borrow reserve; commit to the approved route and configuration; use this account as recipient; satisfy the registry's health-factor band and oracle-derived output floor; and provide nonzero supplied-token, borrowed-token, and health-factor bounds. A requested eMode change must preserve an existing matching category or enter a category from zero while no debt is outstanding.
 - **Impact:** Defines the account identity, relayer policy, collateral contribution, leverage amount, eMode category, swap commitments, and post-open safety bounds for the entire opening life cycle.
- `signature`
 - **Control:** Full control.
 - **Constraints:** Must be a valid ECDSA signature from the immutable EOA owner over the complete `OpenIntent`, this position account's EIP-712 domain, and the current nonce.
 - **Impact:** Authorizes the account to supply collateral and create variable debt

under the exact signed intent.

- routeData
 - **Control:** Full control.
 - **Constraints:** Its hash must equal `intent.conversion.routeDataHash`, and it must decode and validate under the approved route's current market, configuration hash, direction, and slippage limits.
 - **Impact:** Supplies the route-specific market, path commitment, per-hop slippage limit, and execution direction used for the opening swap.

Branches and code coverage

Intended branches

- ☒ A valid owner-signed intent should open a position through an account-owned mode-2 Aave flash loan without prefunded borrow asset.
- ☒ Opening should combine preheld raw collateral with measured route output before supplying to Aave.
- ☒ Opening should enter a signed eMode category from zero and preserve an already matching category across repeated opens.
- ☒ Both a named authorized caller and a permissionless relayer should be able to submit an otherwise valid intent.
- ☒ The account should measure exact route input and output, clear temporary allowance, and enforce supplied-token, debt, and health-factor postconditions.
- ☒ A factory-created account should complete the full open and close life cycle against the pinned Ink Aave deployment.
- ☒ The nonce should increment by one after a successful open.

Negative behavior

- ☒ Revert for an invalid signer, account identity, chain or account domain, nonce, deadline, or authorized caller.
- ☒ Revert for zero or inconsistent amounts, incorrect conversion direction, incorrect recipient, mismatched route data, or a debt token that does not match the borrow reserve.
- ☒ Revert when the registry rejects the route, action, pool, configuration, health-factor floor, execution state, or oracle minimum-output floor.
- ☒ Revert atomically when the route underdelivers, overpulls, reports misleading telemetry, changes an unexpected balance, or attempts account reentrancy.
- ☒ Revert for a non-Pool callback, incorrect initiator, incorrect callback arrays, incorrect asset or amount, nonzero mode-2 premium, or tampered callback context.
- ☒ Revert when final supplied collateral is too low, final variable debt is too high, or the final health factor is below the signed minimum.
- ☐ Revert atomically when Aave rejects a requested eMode category during opening.

Function: setEModeCategory(uint8 categoryId)

This owner-only function changes the position account's Aave eMode category while the account has no outstanding variable debt, preventing an existing leveraged position from being moved underneath different liquidation parameters.

Inputs

- categoryId
 - **Control:** Full control.
 - **Constraints:** The caller must be the immutable account owner, the account's borrow-reserve variable-debt balance must be zero when a debt token exists, and Aave must accept the requested category. Zero disables eMode.
 - **Impact:** Selects the Aave efficiency-mode risk configuration used for future position life cycles.

Branches and code coverage**Intended branches**

- ☒ The owner should set a nonzero eMode category when the account has zero debt.
- ☒ The owner should reset the category to zero after debt has been cleared.
- ☒ The owner should be allowed to set eMode when the reserve reports no variable-debt token.

Negative behavior

- ☒ Revert when the caller is not the immutable owner.
- ☒ Revert when the account has outstanding variable debt.
- ☐ Revert atomically when Aave rejects an unsupported or incompatible category.

5.4. Module: TydroPositionAccountFactory**Function: createAccount(address owner, address pool, address supplyAsset, address borrowAsset, bytes32 userPositionSalt)**

This permissionless factory function deploys the deterministic nonupgradable position-account clone for the supplied immutable identity or returns the existing account at that address when it has already been created.

Inputs

- owner

- **Control:** Full control.
 - **Constraints:** Must not be the zero address.
 - **Impact:** Becomes the clone's immutable EIP-712 signer and direct owner-only authority.
- pool
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and contain deployed code.
 - **Impact:** Becomes the clone's immutable Aave Pool and sole authorized flash-loan callback caller.
- supplyAsset
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and differ from borrowAsset.
 - **Impact:** Becomes the account's immutable collateral asset.
- borrowAsset
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and differ from supplyAsset.
 - **Impact:** Becomes the account's immutable variable-debt and flash-loan asset.
- userPositionSalt
 - **Control:** Full control.
 - **Constraints:** May be any bytes32 value.
 - **Impact:** Selects a distinct deterministic account within the same owner and market namespace.

Branches and code coverage

Intended branches

- ☒ A caller should deploy a clone at the previously predicted address and mark it as a factory-created account.
- ☒ The clone should expose the exact immutable owner, Pool, asset pair, user salt, implementation version, and account-specific EIP-712 domain.
- ☒ Repeated creation with the same identity should return the existing account without changing its identity.
- ☐ Repeated creation should be explicitly verified not to emit a second AccountCreated event.
- ☒ A caller distinct from the position owner should be able to create the account, demonstrating permissionless deployment.
- ☒ Deployment of a new clone should emit AccountCreated with the exact account identity.

Negative behavior

- ☑ Revert before deployment for any zero identity address, a codeless Pool, or identical supply and borrow assets.
- ☑ Direct use of the shared implementation without clone immutable arguments should fail closed.

Function: `predictAccount(address owner, address pool, address supplyAsset, address borrowAsset, bytes32 userPositionSalt)`

This function computes the deterministic CREATE2 address of the nonupgradable position-account clone for an exact owner, live chain, Aave Pool, ordered asset pair, user salt, implementation version, and factory.

Inputs

- `owner`
 - **Control:** Full control.
 - **Constraints:** Must not be the zero address.
 - **Impact:** Selects the immutable signing authority and owner-only recipient embedded in the predicted account.
- `pool`
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and contain deployed code.
 - **Impact:** Selects the immutable Aave Pool that may call the account callback and hold its lending position.
- `supplyAsset`
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and differ from `borrowAsset`.
 - **Impact:** Selects the immutable collateral asset and opening swap output.
- `borrowAsset`
 - **Control:** Full control.
 - **Constraints:** Must be nonzero and differ from `supplyAsset`.
 - **Impact:** Selects the immutable debt asset and opening flash-loan principal.
- `userPositionSalt`
 - **Control:** Full control.
 - **Constraints:** May be any `bytes32` value.
 - **Impact:** Distinguishes multiple otherwise identical accounts for the same owner and market.

Branches and code coverage

Intended branches

- ☒ Prediction should be deterministic and match the manual CREATE2 immutable-argument derivation.
- ☒ The predicted address should equal the address later returned by `createAccount`.
- ☒ Changing the owner, Pool, either asset, user salt, or implementation version should change the account namespace.
- ☐ Changing the live chain ID should change a newly predicted account address for the same remaining inputs.

Negative behavior

- ☐ Revert through `predictAccount` for a zero owner, Pool, supply asset, or borrow asset.
- ☐ Revert through `predictAccount` when the Pool has no deployed code.
- ☐ Revert through `predictAccount` when the supply and borrow assets are identical.

5.5. Module: TydroVelodromeSlipstreamSwapRoute

Function: `acceptOwnership()`

This function completes the route's two-step ownership transfer, grants local path-configuration authority to the nominated address, and clears the pending owner.

Inputs

This function has no inputs.

Branches and code coverage

Intended branches

- ☒ The nominated pending owner should accept ownership and clear the pending-owner slot.

Negative behavior

- ☒ Revert when the caller is not the nominated pending owner.

Function: `executeSwap(TydroPositionTypes.SwapParams params)`

This standalone exact-input route pulls a bounded input from the caller, verifies enabled registry policy and the route-local path commitment, applies an action-specific per-hop Slipstream price limit, executes the configured forward or reversed path, and transfers measured output to the

selected recipient. Direct callers receive no Aave or position-account authority.

Inputs

- `params`
 - **Control:** Full control.
 - **Constraints:** `amountIn`, `minAmountOut`, and `recipient` must be nonzero; the deadline must not have expired; `routeData` must decode to the same market and configuration hash as the outer parameters; the registry market must be enabled and commit to that hash; requested slippage must be between one and the action-specific registry cap; the stored local path hash must match registration; forward or reverse path endpoints must equal `assetIn` and `assetOut`; and the caller must provide allowance and an exact-transfer token whose received amount equals `amountIn`. Every router hop must consume its exact input and produce nonzero output, and the recipient's measured net output must meet `minAmountOut`.
 - **Impact:** Selects the market, assets, exact input, minimum output, recipient, deadline, path direction, and per-hop price movement permitted for the asset transformation.

Branches and code coverage

Intended branches

- ☒ A caller should execute the configured forward opening direction, consuming exact input and delivering measured output to the recipient.
- ☒ A caller should execute the same stored path in reverse for the closing direction.
- ☒ Ambient route input balances should remain untouched.
- ☐ Successful execution should explicitly assert that every temporary route-to-router allowance is cleared.
- ☒ `BoundedSwapExecuted` should report the measured input and output.
- ☒ A superseded route should remain caller-funded and direct-callable while no longer being approved for official account life cycles.

Negative behavior

- ☒ Revert for zero input, zero recipient, zero minimum output, expired deadline, route-data market mismatch, or stale configuration hash.
- ☐ Revert when `routeData` cannot be decoded into the required route execution tuple.
- ☐ Revert when `marketId` identifies an unregistered market.
- ☒ Revert when execution is disabled, the local and registry configuration hashes drift, requested slippage is zero, or requested opening slippage exceeds the configured opening cap.
- ☐ Distinguish the route-level opening and closing slippage caps with different configured

values and reject a closing request that exceeds only the closing cap.

- ☒ Revert when the execution path starts or ends with the wrong asset or a hop returns zero output.
- ☒ Revert when fee-on-transfer or malicious tokens make received input or recipient output differ from the exact measured requirements.
- ☒ Revert when a router consumes only part of its approved hop input, recipient balance decreases, route output balance changes unexpectedly, or final measured output is below the minimum.
- ☐ A successful multihop forward and reverse swap should be covered end-to-end rather than only multihop configuration rejection.

Function: rotateSwapPathConfig(address supplyToken, address borrowToken, SwapPathConfig config, bytes32 expectedOldConfigHash)

This owner-only function replaces a registered market's local Slipstream path while official execution is disabled. It validates the replacement path and checks the registry's current configuration commitment before writing hops and advancing the local configuration epoch.

Inputs

- supplyToken
 - **Control:** Full control.
 - **Constraints:** Must be the nonzero market supply token, differ from borrowToken, and equal the replacement path's final output token.
 - **Impact:** Participates in deriving the directional market whose local path is replaced.
- borrowToken
 - **Control:** Full control.
 - **Constraints:** Must be the nonzero market borrow token, differ from supplyToken, and equal the replacement path's first input token.
 - **Impact:** Participates in deriving the directional market whose local path is replaced.
- config
 - **Control:** Full control.
 - **Constraints:** Must satisfy the same nonempty, continuity, Pool, router, factory, token, and tick-spacing validation as initial configuration.
 - **Impact:** Replaces the exact forward path used by future opening swaps and reversed closing swaps.
- expectedOldConfigHash
 - **Control:** Full control.

- **Constraints:** Must equal the current registry-side configuration commitment, and registry execution must be disabled.
- **Impact:** Prevents a stale path-rotation operation from proceeding against an unexpected registry configuration.

Branches and code coverage

Intended branches

- ☒ The owner should rotate a valid path while the market is disabled and advance the local epoch.
- ☒ A route-local write should invalidate queued activation until governance updates the registry commitment.
- ☒ Cycling local configuration back to an old hash should not revive an old activation snapshot because the local epoch remains monotonic.
- ☒ The governance replacement batch should rotate configuration and update the corresponding registry state before separate activation.

Negative behavior

- ☒ The shared token-pair validator used by `rotateSwapPathConfig` should reject zero or identical supply and borrow tokens.
- ☐ Revert through `rotateSwapPathConfig` when the derived market is not registered.
- ☒ Revert when the market is currently enabled.
- ☐ Revert through `rotateSwapPathConfig` for each invalid replacement-path condition enforced by initial path validation.
- ☒ Revert when `expectedOldConfigHash` does not equal the registry's current commitment.
- ☐ Revert when the caller is not the route owner.

Function: `setSwapPathConfig(address supplyToken, address borrowToken, SwapPathConfig config)`

This owner-only function installs the initial forward Slipstream path from a market's borrow token to its supply token before the directional market is registered. It validates every pool, router, factory, token, tick-spacing, and path-continuity relationship before storing the hops and advancing the local configuration epoch.

Inputs

- `supplyToken`
- **Control:** Full control.

- **Constraints:** Must be nonzero, differ from borrowToken, and equal the final hop's output token.
 - **Impact:** Defines the market collateral asset and forward-path destination.
- borrowToken
 - **Control:** Full control.
 - **Constraints:** Must be nonzero, differ from supplyToken, and equal the first hop's input token.
 - **Impact:** Defines the market debt asset and forward-path source.
- config
 - **Control:** Full control.
 - **Constraints:** Must contain at least one continuous hop. Every hop must use nonzero and distinct tokens, nonzero factory, router, and pool addresses, match the Pool's token pair and tick spacing, match both Pool and router factories, and resolve to the same Pool through the factory. The market must not already be registered.
 - **Impact:** Defines the exact ordered venues and assets used by future forward opens and reversed closes and determines the route-local configuration hash committed by the registry.

Branches and code coverage

Intended branches

- ☒ The owner should store a validated path and expose its canonical hash and incremented local epoch.
- ☒ A valid path should be installed immediately before disabled registry registration in the governance deployment sequence.

Negative behavior

- ☒ Revert when the caller is not the route owner.
- ☒ The shared token-pair validator used by setSwapPathConfig should reject zero or identical supply and borrow tokens.
- ☒ Revert for an empty path, incorrect first or last token, or broken multihop continuity.
- ☒ Revert for a zero hop address, mismatched Pool tokens, incorrect tick spacing, incorrect Pool or router factory, or incorrect factory Pool resolution.
- ☒ Revert when initial configuration is attempted after the market has already been registered.

Function: transferOwnership(address newOwner)

This owner-only function nominates a successor for control of the route's local Slipstream path configuration while leaving configuration authority with the current owner until acceptance.

Inputs

- `newOwner`
 - **Control:** Full control.
 - **Constraints:** The caller must be the current route owner, and the nominee must not be the zero address.
 - **Impact:** Selects the only address permitted to complete the handoff and gain path-installation and path-rotation authority.

Branches and code coverage

Intended branches

- ☒ The current owner should nominate a pending owner without immediately transferring authority.

Negative behavior

- ☒ Revert when the nominee is the zero address.
- ☐ Revert when the caller is not the current route owner.

6. Assessment Results

During our assessment on the scoped Multiply contracts, we discovered five findings. No critical issues were found. One finding was of medium impact and the other findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.